



01211373 Machine Learning and Programming for Industry

Ensemble Learning

Week 4 — Classical Machine Learning

Last Week → This Week

WEEK 3 RECAP

- Regression predicts a number; use RMSE / R^2 to evaluate
- Polynomial degree trades bias for variance
- Ridge & Lasso regularize — and need scaled features
- Random forest regression: no scaling, handles nonlinearity

TODAY

- Why combining many models beats one
- Bagging and random forests, revisited as classifiers
- Boosting: gradient boosting and XGBoost
- A brief word on AdaBoost, for historical context
- Feature importance for root-cause analysis

Today's Running Example: Predictive Maintenance

From vibration, temperature, and running hours, predict a machine's remaining-useful-life risk category before it fails.

Low Risk

Healthy vibration & temperature; keep running normal schedule

Medium Risk

Early warning signs; schedule an inspection soon

High Risk

Strong failure signature; plan maintenance now

Every ensemble method today will be trained on this three-class task, and you'll build all of them yourself in the lab.

Why Ensembles? Wisdom of the Crowd



A single decision tree (Week 2) is fast and interpretable, but tends to overfit and can be unstable — change the data slightly and you can get a very different tree.

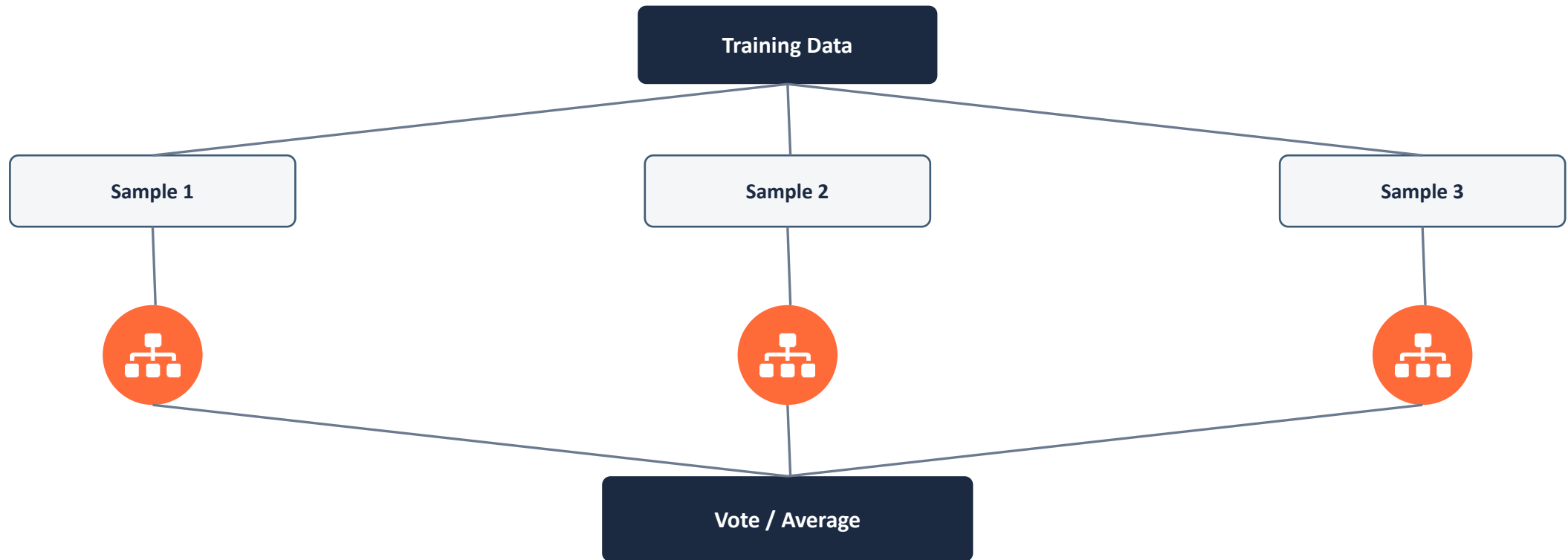
The core idea behind every method today

Train many individually-imperfect models, then combine their predictions — by voting or averaging (bagging), or by having each one correct the last one's mistakes (boosting). The combination is consistently more accurate and more stable than any single model.

Bagging: Bootstrap Aggregating



Draw many random samples (with replacement) from the training data, fit a tree on each, then vote or average.



Random Forests: Bagging + Feature Randomness



Everything from bagging, plus one more trick: each split only considers a random subset of features.

Why bother? If one feature is a strong predictor, plain bagging still builds many trees that all split on it first — the trees end up correlated, which limits how much averaging helps. Forcing each split to ignore most features decorrelates the trees.

```
from sklearn.ensemble import RandomForestClassifier
```

```
RandomForestClassifier(n_estimators=200, max_depth=6)
```

Boosting: Learning from Mistakes, Sequentially



Instead of training trees independently and averaging, boosting trains them one after another — each new tree focuses on what the ensemble so far got wrong.



The learning rate controls how much each new tree is allowed to correct — small steps, many trees, usually generalizes better than a few large steps.

Gradient Boosting



Each new tree is fit to the gradient of the loss — in plain terms, to how (and how much) the current ensemble is wrong on each example.

Strengths

- Often the strongest off-the-shelf accuracy
- Handles nonlinearity and feature interactions
- No scaling needed

Watch out for

- More hyperparameters to tune than a random forest
- Can overfit with too many trees / too high a learning rate
- Trains sequentially — slower than bagging to fit

XGBoost: Gradient Boosting, Engineered



The same gradient boosting idea, with engineering that made it the default choice in industry and competitions for years.

- Built-in L1/L2 regularization on tree weights
- Handles missing values natively — no imputer required
- Highly optimized for speed, parallelized tree building

```
import xgboost as xgb
```

```
xgb.XGBClassifier(n_estimators=200, max_depth=4, learning_rate=0.1)
```

A Quick Word on AdaBoost



The original boosting algorithm (1997) — it reweights misclassified examples after each round instead of fitting residuals directly.

Why we're not spending lab time on it: gradient boosting (and XGBoost) generalize the same core idea, are more flexible about the loss function, and are what you'll actually encounter in industrial ML pipelines today. Worth knowing AdaBoost exists and roughly how it works — not worth building from scratch in 2026.

```
from sklearn.ensemble import AdaBoostClassifier    # good to recognize, rarely your first choice now
```

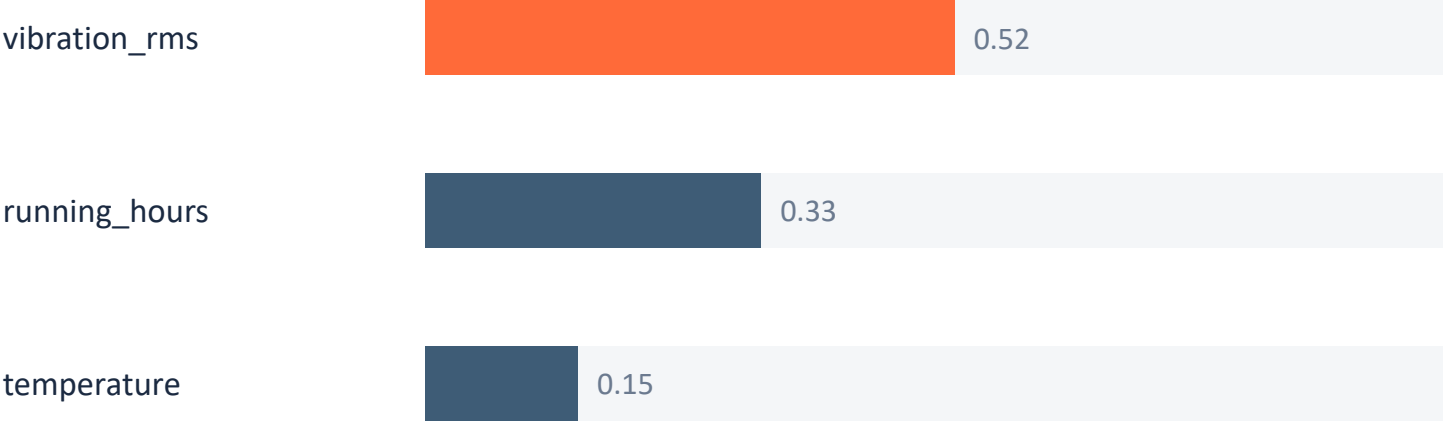
Bagging vs. Boosting

	Bagging / Random Forest	Boosting (GB / XGBoost)
Trees trained	Independently, in parallel	Sequentially, each fixing the last
Main goal	Reduce variance	Reduce bias (and variance, carefully)
Overfitting risk	Low — more trees rarely hurts	Higher — watch n_estimators & learning rate
Typical accuracy	Strong, reliable baseline	Often the best, if well-tuned

Feature Importance: Root-Cause Analysis



Every tree-based ensemble can report which features it relied on most — turning a “black box” prediction into an engineering insight.



Reading this chart: vibration is doing most of the work in this model. That's a concrete, actionable finding — it tells a maintenance engineer exactly which sensor to trust most.

Choosing an Ensemble Method

	Random Forest	Gradient Boosting	XGBoost
Good default?	Yes, very robust	Yes, if tuned	Yes, fast & strong
Needs tuning?	Minimal	Yes — learning rate, depth	Yes — similar, more options
Handles missing values?	No (impute first)	No (impute first)	Yes, natively
Training speed	Fast (parallel)	Slower (sequential)	Fast (optimized)

Today's Lab: Predictive Maintenance, End to End

1

Baseline vs. bagging

A single tree, then a BaggingClassifier of 50 trees.

2

Random forest + feature importance

Fit, then extract and plot which sensors matter most.

3

Gradient boosting & XGBoost

Fit both, compare accuracy against everything so far.

4

Final comparison

One chart, five models — pick your production model.

Key Takeaways

- ✓ Combine many imperfect models; the ensemble beats any one of them.
- ✓ Bagging reduces variance in parallel; boosting reduces bias in sequence.
- ✓ Feature importance turns a tree ensemble into a root-cause analysis tool.

NEXT WEEK

Dimensionality Reduction & Clustering

PCA and k-means / DBSCAN — unsupervised anomaly detection when labels are scarce.